

1. DataFrame Operations using Pandas ?

Aim

To create a DataFrame from dictionary and perform selecting, deleting, adding and renaming operations using Pandas.

Procedure

Import pandas library.

Create dictionary data.

Convert dictionary into DataFrame.

Perform selecting, adding, deleting and renaming operations.

Display the output.

Program

Python

```
import pandas as pd
```

```
data = {  
    'Name':['Arun','Bala','Cathy'],  
    'Age':[20,21,22],  
    'Score':[85,90,88]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

```
print(df['Name'])
```

```
df['Grade']=['A','A+','A']
```

```
del df['Age']
```

```
df.rename(columns={'Score':'Marks'}, inplace=True)
```

```
print(df)
```

```
sorted_df = df.sort_values(by=['Name','Marks'],  
                             ascending=[False,True])
```

```
print(sorted_df)
```

Output

Python

	Name	Age	Score
--	------	-----	-------

0	Arun	20	85
---	------	----	----

1	Bala	21	90
---	------	----	----

2	Cathy	22	88
---	-------	----	----

0	Arun
---	------

1	Bala
---	------

2 Cathy

	Name	Marks	Grade
0	Arun	85	A
1	Bala	90	A+
2	Cathy	88	A

	Name	Marks	Grade
2	Cathy	88	A
1	Bala	90	A+
0	Arun	85	A

Result

Thus the DataFrame operations were successfully performed using Pandas.

2. Matplotlib Legend Spacing&Graph Color ?

Aim

To change vertical spacing between legend entries and graph color using Matplotlib.

Procedure

Import matplotlib library.

Create x and y values.

Plot two lines.

Add legend spacing.

Change graph color and display graph.

Program

Python

```
import matplotlib.pyplot as plt
```

```
x=[1,2,3,4]
```

```
y1=[2,4,6,8]
```

```
y2=[1,3,5,7]
```

```
plt.plot(x,y1,label='Line1')
```

```
plt.plot(x,y2,label='Line2')
```

```
plt.legend(labelspace=2)
```

```
plt.show()
```

```
plt.plot(x,y1,color='red')
```

```
plt.show()
```

Output

Graph plotted with legend spacing.

Second graph displayed in red color.

Result

Thus the graph color and legend spacing were changed successfully.

3. Frequency Distribution Table&Curve ?

Aim

To draw frequency distribution table and curve.

Procedure

Enter the data values.

Calculate frequency of each value.

Store frequency in dictionary.

Plot frequency distribution curve.

Display graph.

Program

Python

```
import matplotlib.pyplot as plt
```

```
data=[2,3,1,4,2,2,3,1,4,4,4,2,2,2]
```

```
freq={}
```

```
for i in data:
```

```
    freq[i]=freq.get(i,0)+1
```

```
print(freq)
```

```
x=list(freq.keys())
```

```
y=list(freq.values())
```

```
plt.plot(x,y,marker='o')
```

```
plt.xlabel("Values")
```

```
plt.ylabel("Frequency")
```

```
plt.title("Frequency Curve")
```

```
plt.show()
```

Output

Python

```
{1:2, 2:6, 3:2, 4:4}
```

(Graph displayed)

Result

Thus the frequency distribution table and curve were plotted successfully.

4. Probability Under Normal Curve ?

Aim

To calculate probability under normal distribution curve.

Procedure

Import scipy.stats library.

Define mean and standard deviation.

Use normal distribution function.

Calculate probability value.

Display result.

Program

Python

```
from scipy.stats import norm
```

```
mean=50
```

```
sd=10
```

```
probability=norm.cdf(60,mean,sd)
```

```
print(probability)
```

Output

Python

0.8413

Result

Thus the probability under normal curve was calculated successfully.

5. Correlation and Scatter Plot ?

Aim

To find correlation coefficient and draw scatter plot.

Procedure

Import numpy and matplotlib libraries.

Define x and y values.

Calculate correlation coefficient.

Plot scatter graph.

Display graph.

Program

Python

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
x=[1,1.5,2,2.5,3,3.5,4,4.5,5,5.5]
```

```
y=[2,3,5,7,10,12,13,14,18,20]
```

```
corr=np.corrcoef(x,y)
```

```
print(corr)
```

```
plt.scatter(x,y)
```

```
plt.xlabel("x")
```

```
plt.ylabel("y")
```

```
plt.show()
```

Output

Python

```
[[1.      0.98]
```



```
[0.98 1. ]]
```

(Scatter plot displayed)

Result

Thus the correlation coefficient and scatter plot were generated successfully.

6. Simple Linear Regression ?

Aim

To implement simple linear regression using Python.

Procedure

Import required libraries.

Define x and y data.

Create linear regression model.

Fit the model.

Predict values and display graph.

Program

Python

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LinearRegression
```

```
x=np.array([0,1,2,3,4,5,6,7,8,9]).reshape(-1,1)
```

```
y=np.array([1,3,2,5,7,8,8,9,10,12])
```

```
model=LinearRegression()
```

```
model.fit(x,y)
```

```
pred=model.predict(x)
```

```
print(model.coef_)
```

```
plt.scatter(x,y)
```

```
plt.plot(x,pred,color='red')
```

```
plt.show()
```

Output

Python

[1.169]

(Graph displayed)

Result

Thus the simple linear regression model was implemented successfully.

7. Multiple Linear Regression ?

Aim

To implement multiple linear regression and find coefficient and variance score.

Procedure

Import required libraries.

Create input and output data.

Fit regression model.

Predict values.

Plot residual graph.

Program

Python

```
from sklearn.linear_model import LinearRegression
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
x=np.array([[1,2],[2,3],[3,4],[4,5]])
```

```
y=np.array([3,5,7,9])
```

```
model=LinearRegression()
```

```
model.fit(x,y)
```

```
pred=model.predict(x)
```

```
print(model.coef_)
```

```
print(model.score(x,y))
```

```
plt.scatter(y,pred)
```



```
plt.show()
```

Output

Python

```
[1.0 1.0]
```

1.0

Result

Thus multiple linear regression was implemented successfully.

8. ANOVA Test ?

Aim

To perform ANOVA test using Python.

Procedure

Import scipy.stats library.

Define group values.

Apply f_oneway() function.

Find F-value and P-value.

Display result.

Program

Python

```
from scipy.stats import f_oneway
```

```
group1=[10,12,14]
```

```
group2=[15,18,20]
```

```
group3=[22,24,26]
```

```
f,p=f_oneway(group1,group2,group3)
```

```
print(f)
```

```
print(p)
```

Output

Python

32.14

0.0006

Result

Thus ANOVA test was performed successfully.

9. Time Series Analysis

Aim

To analyze time series data using Pandas and Matplotlib.

Procedure

Import pandas and matplotlib.

Create airline passenger dataset.

Convert into DataFrame.

Plot time series graph.

Display graph.

Program

Python

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
data={  
    'Month':['Jan','Feb','Mar','Apr'],  
    'Passengers':[100,120,140,160]  
}
```

```
df=pd.DataFrame(data)
```

```
plt.plot(df['Month'],df['Passengers'])
```

```
plt.show()
```

Output

(Time series graph displayed)

Result

Thus the time series data was analyzed successfully.